



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**EFFICIENT SUDOKU GENERATION AND SOLVING USING RECURSIVE
BACKTRACKING**

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements for the
Course of*

CSA0603 & DESIGN AND ANALYSIS OF ALGORITHMS

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Aksharaa.B (192311278)

Akshaya A (192421222)

Jeslet Jessica (192524135)

Gayathri N P (192421362)

Under the Supervision of

Dr. Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**EFFICIENT SUDOKU GENERATION AND SOLVING USING RECURSIVE BACKTRACKING**” has been carried out by **Akshaya A (192421222)** under the supervision of **Dr. Solainayagi** and is submitted in partial fulfilment of the requirements for the current semester of the program Computer Science And Engineering at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.G.Charlyn Pushpa Latha

Professor

Computer Science And Engineering

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr Sarala V

Professor

Computer Science And Engineering

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

I **Akshaya A (192421222)** of the Computer Science And Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled **“EFFICIENT SUDOKU GENERATION AND SOLVING USING RECURSIVE BACKTRACKING”** is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:

Date:

Signature of the Students with Names

Akshaya A (192421222)

INDIVIDUAL CONTRIBUTION STATEMENT

The following allocation is a proposed project-role distribution for documentation. The team should revise the percentages if the actual contribution differs.

Student Name / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. Contribution (%)
Aksharaa. B / 192311278	Sudoku generation and system development	Puzzle generation and algorithm implementation	Puzzle testing	Project documentation	25%
Akshaya A / 192421222	Sudoku solving module	Recursive Backtracking solver	Solver testing and analysis	Report preparation	25%
Jeslet Jessica / 192524135	Validation and performance analysis	Validation module	Performance evaluation	Documentation	25%
Gayathri N P / 192421362	User interface and visualization	GUI development	Interface testing	Presentation and report	25%
Total					100%

ABSTRACT

Sudoku is a logic-based number placement puzzle in which numbers from 1 to 9 must be arranged such that each number occurs only once in every row, column and 3×3 sub-grid. Solving difficult Sudoku puzzles manually can require considerable time and may involve repeated trial-and-error.

This project proposes an Efficient Sudoku Generation and Solving System using Recursive Backtracking. The system is designed to generate Sudoku puzzles, solve them systematically, validate the generated solutions and provide an interactive visualization of the complete process.

The main algorithm used in the project is Recursive Backtracking. The algorithm searches for an empty cell and attempts candidate numbers from 1 to 9. Before placing a number, the system checks whether the candidate satisfies the row, column and 3×3 sub-grid constraints. If a valid number is placed, the algorithm recursively continues to the next empty cell. When a conflict occurs, the algorithm backtracks to the previous state and tries another candidate.

The project is divided into four major modules: Puzzle Generation, Sudoku Solver using Recursive Backtracking, Puzzle Validation & Performance Analysis, and User Interface & Visualization. Performance is evaluated using execution time, number of recursive calls, puzzle success rate and comparison across different difficulty levels.

The project demonstrates practical applications of recursion, backtracking, Depth-First Search and constraint checking in solving a constraint-based problem.

Keywords: Sudoku, Recursive Backtracking, Recursion, Depth-First Search, Constraint Checking, Puzzle Generation, Algorithm Analysis.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS / SYMBOLS	viii
1	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	10
2	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	14
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	17
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	22
5	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	28
	REFERENCES	31
	APPENDICES	33

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Conceptual view of dust accumulation and visible module defects	2
3.1	Proposed computer-vision system architecture	9
4.1	Conceptual model output for localized dust and defect regions	15
4.2	Recommended evaluation metrics	17
4.3	Indicative project milestone timeline	19

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparison of existing PV inspection approaches	5
3.1	Stakeholder and user requirements	8
3.2	Functional and non-functional requirements	8
3.3	Constraints and applicable standards	9
3.4	Design specifications	10
3.5	Alternative solution evaluation matrix	10
3.6	Trade-off analysis	11
3.7	Sustainability, ethics, safety and security assessment	12
3.8	Project risk register	12
4.1	Software and computational resources	14
4.2	Test plan and verification criteria	16
4.3	Result-recording template	17
4.4	Objective achievement matrix	18
4.5	Roles and budget framework	19

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	—
CNN	Convolutional Neural Network	—
CV	Computer Vision	—
EL	Electroluminescence	—
FN	False Negative	count
FP	False Positive	count
F1	F1-score	—
IoU	Intersection over Union	—
PV	Photovoltaic	—
RGB	Red–Green–Blue	—
ROI	Region of Interest	—
TP	True Positive	count
UAV	Unmanned Aerial Vehicle	—
mAP	mean Average Precision	—

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Sudoku is a logic-based number placement puzzle consisting of a 9×9 grid. The objective is to fill every empty cell with numbers from 1 to 9 while ensuring that each number appears only once in every row, column and 3×3 sub-grid.

Solving Sudoku computationally requires a systematic method for exploring possible values while maintaining the constraints of the puzzle. The **Recursive Backtracking algorithm** provides an effective approach because it can make a temporary choice, continue recursively and return to an earlier decision whenever the selected choice produces a conflict.

The proposed project develops an integrated Sudoku system capable of generating puzzles, solving them using Recursive Backtracking, validating solutions and displaying the results through an interactive interface.

1.2 Need for the Project

Generating and solving Sudoku puzzles can become challenging when the puzzle contains a large number of empty cells.

The need for the project arises from the following problems:

- Manual Sudoku solving can be time-consuming.
- Difficult puzzles may require extensive trial-and-error.
- Invalid number placements need to be detected efficiently.
- Existing simple approaches may not provide integrated generation, solving and validation.
- Performance needs to be measured to understand algorithm efficiency.
- A user-friendly visualization can make the algorithm easier to understand.

The PPT identifies the need for an efficient system capable of generating, solving, validating and visualizing Sudoku puzzles.

1.3 Problem Statement

Generating valid Sudoku puzzles and solving difficult Sudoku configurations are challenging computational problems. Manual solving can be time-consuming and prone to errors, while traditional trial-and-error methods may perform unnecessary searches.

There is therefore a need for an efficient system that can:

1. Generate valid Sudoku puzzles.
2. Solve incomplete puzzles systematically.
3. Check Sudoku constraints automatically.
4. Backtrack when an invalid configuration is encountered.
5. Validate the final solution.
6. Analyse the performance of the solving algorithm.
7. Provide a simple interface for users.

1.4 Project Aim

The aim of this project is to develop an efficient Sudoku generation and solving system using **Recursive Backtracking** to systematically explore valid solutions, reduce unnecessary searches and provide a simple interactive Sudoku application.

1.5 Project Objectives

The objectives of the project are:

1. To develop an efficient system for generating valid Sudoku puzzles.
2. To implement the Recursive Backtracking algorithm for Sudoku solving.
3. To reduce unnecessary searches using constraint checking and pruning.
4. To analyse the time and space complexity of the implemented algorithm.
5. To support different Sudoku puzzle difficulty levels.
6. To demonstrate DAA concepts such as recursion, backtracking and optimization.
7. To validate Sudoku puzzles and their solutions.
8. To provide performance analysis using execution time and recursive calls.

These objectives are directly based on the objectives provided in the PPT.

1.6 Scope

Included

- Sudoku puzzle generation.
- 9×9 Sudoku grid.
- Recursive Backtracking solver.
- Candidate number generation.
- Constraint checking.
- Puzzle validation.
- Performance analysis.
- Difficulty-level support.
- Interactive graphical interface.
- Generate, Solve, Validate and Clear operations.

Excluded

- Large-scale distributed Sudoku solving.
- Advanced machine-learning-based solving.
- Image-based Sudoku recognition.
- Automatic online puzzle retrieval.
- Advanced heuristic solving in the current implementation.

Assumptions

- The Sudoku grid follows a 9×9 structure.
- Each cell contains either a valid number or an empty value.
- Sudoku rules are maintained throughout the solving process.
- The generated puzzle can be passed to the solving module.

Boundary Conditions

The current implementation focuses on Sudoku generation, recursive solving, validation, visualization and basic performance analysis.

1.7 Expected Engineering Outcome

The expected outcome is an interactive Sudoku application capable of generating a puzzle, solving it using Recursive Backtracking, validating the solution and presenting the results to the user.

The solver should identify empty cells, test candidate numbers, apply Sudoku constraints and perform backtracking when necessary. The system should also provide performance information such as execution time and recursive calls.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant studies and existing approaches related to Sudoku solving, recursive backtracking, constraint-based search and alternative solving algorithms were reviewed.

The approaches are compared based on their solving strategy, computational requirements, advantages, limitations and suitability for the proposed Sudoku system.

2.2 Review of Existing Work

Sudoku solving has been studied using different algorithmic approaches. Recursive Backtracking is one of the commonly used techniques because it systematically explores possible assignments while reversing decisions that lead to invalid configurations.

Other approaches can use techniques such as constraint propagation, heuristic search and metaheuristic algorithms. These approaches may improve solving efficiency for particular puzzle configurations but can also introduce additional algorithmic complexity.

The proposed project selects Recursive Backtracking because it clearly demonstrates recursion, systematic search and backtracking while providing an understandable implementation suitable for the project objectives.

The PPT references studies comparing backtracking with approaches such as simulated annealing, alternating projection, constraint propagation and Dancing Links.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance
Manual Solving	Simple for humans	Time-consuming for difficult puzzles	Low
Recursive Backtracking	Systematic and easy to implement	May require large search for difficult puzzles	Highly Relevant

Existing Approach	Advantages	Limitations	Relevance
Constraint Propagation	Reduces candidate space	More complex implementation	Relevant
Metaheuristic Methods	Can solve complex configurations	Computationally more complex	Provides comparison
Dancing Links	Efficient exact-cover approach	More complex implementation	Alternative

2.4 Research / Engineering Gap

Existing Sudoku-solving techniques provide different levels of efficiency and complexity. Advanced approaches may require more sophisticated data structures, heuristics or optimization techniques.

There is a need for a simple and understandable algorithmic approach that can demonstrate how a constraint-based puzzle can be solved systematically.

The proposed project addresses this requirement using **Recursive Backtracking**, combined with constraint checking and performance analysis.

2.5 Proposed Contribution

The proposed project develops an integrated Sudoku application with four modules:

1. **Module 1 – Puzzle Generation**
2. **Module 2 – Sudoku Solver using Recursive Backtracking**
3. **Module 3 – Puzzle Validation & Performance Analysis**
4. **Module 4 – User Interface & Visualization**

Module 2 forms the main solving component by exploring possible values recursively and backtracking whenever a conflict occurs. Module 3 evaluates the correctness and performance of the solution, while Module 4 provides an interactive visualization.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements		
Stakeholder	Requirement	
User	Generate Sudoku puzzles easily	
User	Select puzzle difficulty	
User	Solve Sudoku automatically	
User	Validate a Sudoku solution	
User	View solving results	
Student/Developer	Analyse algorithm performance	

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Generate puzzle	Functional	Valid Sudoku puzzle generated
Select difficulty	Functional	Easy/Medium/Hard available
Find empty cell	Functional	Empty cell identified
Check candidate	Functional	Row, column and sub-grid checked
Solve puzzle	Functional	Valid solution generated
Backtrack	Functional	Incorrect candidate removed
Validate solution	Functional	Sudoku rules verified
Performance	Non-Functional	Execution time and recursive calls measured
Usability	Non-Functional	Simple interactive interface

3.2 Constraints & Applicable Standards

Standard / Requirement	Requirement	Application
Sudoku Rules	No duplicate values in rows	Applied during solving
Sudoku Rules	No duplicate values in columns	Applied during solving
Sudoku Rules	No duplicate values in 3×3 sub-grids	Applied during solving
Algorithmic Constraint	Invalid candidates should not be placed	Constraint checking
Software Design	Modular implementation	Four project modules
Performance	Solver efficiency should be measurable	Execution time and recursive calls

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Grid Size	9×9	Grid	High
Candidate Values	1–9	Number	High
Row Constraint	No duplicate values	—	High
Column Constraint	No duplicate values	—	High
Sub-grid Constraint	No duplicate values	—	High
Solver	Recursive Backtracking	Algorithm	High
Search Method	Depth-First Search	—	High
Performance	Execution time and recursive calls	Measures	Medium

3.3 Alternative Solutions & Evaluation

Alternative 1: Manual Solving

Manual solving depends on human reasoning and can become time-consuming for difficult puzzles.

Alternative 2: Recursive Backtracking

Recursive Backtracking systematically explores candidate values and reverses invalid choices. It is selected because it is suitable for demonstrating recursion and backtracking concepts.

Alternative 3: Advanced Heuristic Methods

Heuristics such as Minimum Remaining Values can reduce the search space, but they introduce additional implementation complexity and are considered as future enhancement.

Criterion	Manual	Recursive Backtracking	Advanced Heuristic
Performance	Low	High	High
Implementation Complexity	Low	Medium	High
Constraint Handling	Manual	Automatic	Automatic
DAA Demonstration	Medium	High	High
Selected	No	Yes	Future

3.4 Selected Design & Detailed Design

The **Recursive Backtracking algorithm** is selected as the core Sudoku-solving technique.

The solver receives an incomplete Sudoku grid and searches for an empty cell. It then attempts possible numbers from 1 to 9. A number is placed only when it satisfies the row, column and sub-grid constraints.

The algorithm recursively continues this process. If no valid candidate can be placed at a later stage, the algorithm backtracks to the previous decision and tries another candidate.

Key Engineering Process

Input Puzzle → Empty Cell → Candidate Number → Constraint Check → Place Number → Recursive Call → Conflict? → Backtrack → Continue → Solved Puzzle

3.4 Trade-off Analysis

Design Decision	Alternative	Benefit	Disadvantage	Final Decision
Recursive solving	Manual solving	Systematic search	May require many recursive calls	Recursive solving
Backtracking	Trial-and-error	Controlled reversal	Can increase search time	Backtracking
Constraint checking	Blind search	Prevents invalid paths	Requires validation at every step	Constraint checking
9 × 9 grid	Variable grid	Standard Sudoku format	Limited to standard Sudoku	9 × 9
Tkinter GUI	Command-line interface	Better visualization	Additional implementation	Tkinter

3.5 Sustainability, Ethics, Safety, Risk & Security

Sustainability

The project is primarily software-based and requires no specialized physical resources. Efficient search and constraint checking help reduce unnecessary computation.

Ethics

The system should produce correct and reproducible results. The project documentation acknowledges the algorithms and references used in the development.

Health & Safety

The system does not directly control physical hardware. Its operation is limited to software-based puzzle generation and solving.

Security

The application performs controlled processing of Sudoku grid data. No sensitive user information is required for the basic system.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Excessive recursive calls	Medium	Medium	Medium	Constraint checking	Low
Invalid puzzle generation	Medium	High	High	Puzzle validation	Medium
Slow solving for difficult puzzle	Medium	Medium	Medium	Performance analysis	Low
GUI input error	Low	Medium	Low	Input validation	Low
Incorrect solution	Low	High	Medium	Solution validation	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project follows a **modular development approach** consisting of four modules.

Module	Purpose
Module 1 – Puzzle Generation	Generates Sudoku puzzles
Module 2 – Sudoku Solver	Solves puzzles using Recursive Backtracking
Module 3 – Validation & Performance Analysis	Checks correctness and measures performance
Module 4 – User Interface & Visualization	Provides interactive Sudoku application

Module 1 initializes the grid, selects empty cells and generates candidate numbers.

Module 2 receives the puzzle and applies Recursive Backtracking.

Module 3 validates rows, columns and sub-grids and measures execution time and recursive calls.

Module 4 provides difficulty selection, Sudoku grid visualization and Generate, Solve, Validate and Clear controls.

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The project is implemented using **Python**. The Sudoku generation, solving and validation operations are handled programmatically.

Tkinter is used to create the graphical user interface, while **NumPy** supports efficient handling of Sudoku grid data.

The main algorithm is Recursive Backtracking. Performance analysis is used to evaluate execution time and recursive calls.

Tools Used

Tool / Technology	Purpose
Python	Core implementation
Tkinter	GUI development
NumPy	Grid processing
Recursive Backtracking	Sudoku solving
Performance Analysis	Solver evaluation

4.2 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion
Puzzle generation	Generate Sudoku	Valid puzzle generated
Empty cell detection	Provide incomplete puzzle	Empty cell identified
Candidate checking	Test candidate values	Invalid candidates rejected
Row validation	Insert duplicate row value	Duplicate rejected
Column validation	Insert duplicate column value	Duplicate rejected
Sub-grid validation	Insert duplicate sub-grid value	Duplicate rejected
Backtracking	Use puzzle requiring reversal	Previous choice reversed
Solving	Provide solvable puzzle	Complete solution generated
Validation	Validate completed puzzle	Correct solution confirmed
GUI	Test buttons	Controls operate correctly

4.3 Results

The implemented system successfully demonstrates the generation and solving of Sudoku puzzles using Recursive Backtracking.

The solver:

- Identifies empty cells.
- Attempts numbers from 1 to 9.
- Checks Sudoku constraints.
- Places valid numbers.
- Continues recursively.
- Backtracks when conflicts occur.
- Produces a complete solution when one exists.

Module 3 evaluates the system using execution time, recursive calls, success rate and puzzle difficulty comparison.

The screenshot displays the 'SUDOKU SOLVER' application window. The title bar reads 'Efficient Sudoku Generation and Solving using Recursive Backtracking'. The main title is 'SUDOKU SOLVER'. Below it, the subtitle is 'Efficient Sudoku Generation and Solving using Recursive Backtracking'. The interface includes a 'Difficulty' dropdown set to 'Medium', and buttons for 'Generate Puzzle', 'Solve Puzzle', 'Validate', and 'Clear'. A 9x9 grid shows the solved puzzle:

1	6	4	5	9	3	8	7	2
9	5	3	7	8	2	6	4	1
7	8	2	4	6	1	9	3	5
3	7	6	9	5	4	1	2	8
4	2	9	6	1	8	7	5	3
5	1	8	2	3	7	4	9	6
8	9	1	3	4	5	2	6	7
6	3	7	1	2	9	5	8	4
2	4	5	8	7	6	3	1	9

Below the grid, the 'OUTPUT / PERFORMANCE ANALYSIS' section shows:

- Status: SOLVED ✓
- Execution Time: 0.030858 seconds
- Recursive Calls: 1258
- Validation: Complete and Valid ✓

A 'Validation' dialog box is open, displaying an information icon and the message 'Sudoku is complete and valid!' with an 'OK' button.

4.4 Data Analysis & Engineering Judgment

The performance of Recursive Backtracking depends on the complexity of the Sudoku puzzle. Puzzles with a greater number of possible candidate combinations can require more recursive calls.

Constraint checking reduces unnecessary searches by preventing invalid values from being explored further.

The following performance parameters are considered:

Metric	Purpose
Execution Time	Measures time required to solve
Recursive Calls	Measures amount of recursive search
Success Rate	Measures successful puzzle solving
Difficulty Comparison	Compares performance across puzzle levels

The PPT specifically identifies these measures for performance analysis.

4.5 Comparison with Existing Solutions & Achievement of Objectives

Comparison

Feature	Manual Solving	Proposed System
Puzzle generation	Manual	Automated
Candidate checking	Manual	Automated
Solving	Human reasoning	Recursive Backtracking
Backtracking	Manual	Automatic
Validation	Manual	Automatic
Performance measurement	Difficult	Supported
Visualization	Limited	Interactive GUI

Achievement of Objectives

Objective	Evidence	Achievement
Generate Sudoku puzzles	Puzzle Generation Module	Fully Achieved
Implement Recursive Backtracking	Sudoku Solver Module	Fully Achieved
Reduce unnecessary searches	Constraint checking	Fully Achieved
Analyse complexity	Performance analysis	Achieved
Support difficulty levels	Difficulty selection	Achieved
Demonstrate DAA concepts	Recursion and backtracking	Fully Achieved

4.6 Strengths & Limitations

Strengths

- Systematically solves Sudoku puzzles.
- Uses Recursive Backtracking.
- Automatically checks Sudoku constraints.
- Performs backtracking when conflicts occur.
- Provides puzzle generation.
- Provides puzzle validation.
- Supports performance analysis.
- Provides an interactive graphical interface.
- Demonstrates important DAA concepts.

Limitations

- Recursive Backtracking may require many searches for difficult puzzles.
- Execution time can increase depending on puzzle complexity.
- The current solver does not implement advanced heuristics such as MRV.
- Unique-solution generation requires further enhancement.

- The current implementation focuses on standard Sudoku.

4.7 Project Planning, Roles & Budget

Project Activities

Activity	Status
Problem identification	Completed
Literature review	Completed
System design	Completed
Puzzle generation	Completed
Recursive Backtracking solver	Completed
Validation module	Completed
Performance analysis	Completed
GUI implementation	Completed
Documentation	Completed

Team Roles

Student	Assigned Responsibility
Aksharaa.B	Puzzle Generation
Akshaya A	Recursive Backtracking Solver
Jeslet Jessica	Validation & Performance Analysis
Gayathri N P	User Interface & Visualization

Budget

Item	Estimated Cost	Actual Cost
Software / Development	₹0	₹0
Computing Resources	₹0	₹0
Documentation	₹0	₹0
Other Resources	₹0	₹0
Total	₹0	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project successfully develops an **Efficient Sudoku Generation and Solving System using Recursive Backtracking**.

The system consists of four major modules covering puzzle generation, Sudoku solving, validation and performance analysis, and user interface visualization.

The Recursive Backtracking algorithm systematically explores possible values for empty cells. It validates each candidate against Sudoku constraints and backtracks whenever a selected value produces an invalid configuration.

The project demonstrates the practical application of important DAA concepts including **recursion, backtracking, Depth-First Search, constraint checking and search-space reduction**.

The modular architecture makes the system easier to understand, test, modify and extend.

5.2 Future Work

The system can be enhanced in the following ways:

Enhanced Puzzle Generation

- Develop a generation method that guarantees a unique solution.
- Introduce advanced difficulty levels.
- Support different Sudoku sizes such as 4×4 , 9×9 and 16×16 .

Improved Solving

- Introduce **Minimum Remaining Values (MRV)** heuristic.
- Compare Recursive Backtracking with other solving algorithms.
- Provide step-by-step visualization of solving and backtracking.

Advanced Features

- Develop web and mobile versions.
- Add player statistics.
- Maintain solving history.
- Add performance tracking.
- Introduce a hint system.
- Add interactive animations.

These future enhancements are directly aligned with the future-work section in the PPT.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

Through this project, the team gained knowledge of:

- Recursive algorithms.
- Backtracking.
- Depth-First Search.
- Constraint checking.
- Sudoku puzzle generation.
- Algorithm performance analysis.
- GUI development.
- Modular software development.

Engineering & Professional Skills Developed

The project helped develop:

- Algorithm design.
- Problem solving.
- Programming.
- System design.

- Testing and debugging.
- Performance analysis.
- Technical documentation.
- Team collaboration.

Individual Reflection

The most important learning from the project was understanding how **Recursive Backtracking** can be applied to a constraint-based problem such as Sudoku. The implementation helped demonstrate how a solution can be explored recursively and how incorrect choices can be reversed through backtracking.

The project also provided practical understanding of how algorithm performance can be evaluated using execution time and recursive calls.

If the system were redesigned, advanced heuristics such as **Minimum Remaining Values** could be introduced to reduce unnecessary searches and improve solver efficiency. The PPT also identifies MRV and step-by-step backtracking visualization as future improvements.

REFERENCES

1. E. C. Chi and K. Lange, **“Techniques for Solving Sudoku Puzzles,”** 2022.
2. B. Sun, X. Sun, Y. Wu, Y. Yin, and G. Yang, **“A New Algorithm for Generating Unique-Solution Sudoku,”** IEEE International Conference on Network Computing and Advanced Information Management (ICNC), 2008, DOI: 10.1109/ICNC.2008.788.
3. T. Yato and T. Seta, **“Complexity and Completeness of Finding Another Solution and Its Application to Puzzles,”** IEICE Transactions on Fundamentals, Vol. E86-A, No. 5, pp. 1052–1060, 2003.
4. S. Hira, N. Bhagwatkar, K. Agrawal, and N. Loya, **“Sudoku Solver: A Comparative Study of Different Algorithms and Image Processing Techniques,”** 2024, DOI: 10.3233/ATDE231062.
5. R. Lewis, **“Metaheuristics can solve Sudoku puzzles,”** Journal of Heuristics, Vol. 13, pp. 387–401, 2007, DOI: 10.1007/S10732-007-9012-8.

These references are the ones listed in your PPT.

APPENDICES

Appendix A – Detailed Algorithms

A.1 Sudoku Generation Algorithm

1. Initialize an empty 9×9 Sudoku grid.
2. Select an empty cell.
3. Generate candidate numbers from 1 to 9.
4. Randomly arrange candidate numbers.
5. Check the validity of each candidate.
6. Place a valid candidate.
7. Continue until the required puzzle is generated.

The PPT describes initialization, empty-cell selection and randomized candidate generation as part of Module 1.

A.2 Sudoku Solving Algorithm

1. Find an empty cell.
2. Try numbers from 1 to 9.
3. Check Sudoku constraints.
4. Place a valid number.
5. Recursively continue.
6. Backtrack when a conflict occurs.
7. Stop when all cells are filled.

Appendix B – System Architecture / Flowchart

Puzzle Generation

↓

Generated Sudoku Puzzle

↓

Find Empty Cell

↓

Generate Candidate Numbers

↓

Check Row / Column / 3×3 Sub-grid

↓

Valid Candidate?

→ No → Try Next Candidate

→ Yes → Place Number

↓

Recursive Call

↓

Conflict?

→ Yes → Backtrack

→ No → Continue

↓

Complete Sudoku

↓

Validation & Performance Analysis

↓

GUI Output

Appendix C – Source Code / Code Repository Information

The complete source code consists of the Sudoku generation, Recursive Backtracking solver, validation, performance analysis and Tkinter-based user interface components.

Only essential code excerpts should be included in the final report, while the complete implementation may be maintained separately in the approved project repository.

Appendix D – Performance Data

The following parameters can be recorded during experimental testing:

Puzzle Difficulty	Execution Time	Recursive Calls	Success
Easy	—	—	—
Medium	—	—	—
Hard	—	—	—

Note: Actual numerical performance values should be filled from the project's executed test results rather than being invented.

Appendix E – Experimental / Raw Data

The raw testing data may include:

- Generated Sudoku grids.
- Solved Sudoku grids.
- Execution time.
- Recursive call count.
- Difficulty level.
- Validation result.
- Number of successful solutions.

Appendix F – Ethics / Institutional Approval

Not applicable to the current software-based Sudoku project unless specifically required by the institution.

Appendix G – Project Meeting / Progress Records

Activity	Status
Problem identification	Completed
Literature study	Completed
Algorithm selection	Completed
Module design	Completed
Implementation	Completed
Testing	Completed
Performance analysis	Completed
Documentation	Completed

Appendix H – User / Industry Feedback

Where applicable, feedback regarding the usability of the Sudoku interface, clarity of visualization and solver performance may be recorded.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Where applicable.

Appendix J – Individual Contribution Evidence

Individual contribution records may include:

- Module development records.
- Testing contribution.
- Documentation contribution.
- Presentation contribution.
- Source-code contribution.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

Outcome	Evidence	SO / Area	Location in Report
Complex engineering problem formulation	Sudoku problem definition	SO1	Chapter 1
Engineering design under constraints	Sudoku constraints and algorithm design	SO2	Chapter 3
Written/oral technical communication	Complete technical report and presentation	SO3	Whole Report
Ethics and professional responsibility	Declaration and responsible development	SO4	Chapter 3
Teamwork and leadership	Individual contribution and project roles	SO5	Chapter 4
Experimentation, analysis and engineering judgment	Performance analysis and testing	SO6	Chapter 4
Independent acquisition of new knowledge	Learning recursion and backtracking	SO7	Chapter 5
Standards / constraints	Sudoku rules and software requirements	SO2	Chapter 3
Sustainability and societal impact	Efficient computational processing	SO2/SO4	Chapter 3
Risk assessment and mitigation	Risk register	SO2/SO4	Chapter 3
Security considerations	Controlled software processing	Relevant SO	Chapter 3
Integrated/system-level engineering	Four-module Sudoku system	SO1/SO2	Chapters 3–4

Outcome	Evidence	SO / Area	Location in Report
Project planning and management	Activity plan and team roles	SO5	Chapter 4
Individual contribution	Contribution statement	SO1–SO7	Contribution Statement + Chapter 4

The structure follows the outcome-mapping approach used in the supplied report template.

Important bruh: I have deliberately **not invented actual execution-time values, recursive-call counts, screenshots, or experimental percentages**. Your PPT says those are performance measures, but it doesn't provide numerical results. Those should come from your actual program execution.

Also, the **future enhancement section in your PPT specifically mentions unique-solution generation, MRV, different grid sizes, comparison with other algorithms, web/mobile versions, statistics, hints and animations**, so I kept those rather than replacing them with unrelated content.